

PROJECT EXPLANATION & ARCHITECTURE GUIDE

AI Chatbot for College — Everything Explained in the Easiest Way

⚡ WHAT IS THIS PROJECT IN 30 SECONDS?

Imagine having a super-smart college receptionist who never sleeps, never gets tired, speaks and listens in English, remembers every student's attendance and fee status, and answers college questions instantly. That is exactly what this project is! It is a **full-stack, voice-enabled AI Campus Chatbot and Student Portal** built specifically for GH Raisoni College using **Python (Flask)**, **SQLite**, and modern **Web Speech / PWA technologies**.

1. THE REAL-WORLD PROBLEM WE SOLVED

✗ The Traditional College Way	✓ Our AI Chatbot Solution
Students wait in long physical queues at the admin/accounts desk for minor inquiries (fee dues, exam dates).	Instant 24/7 web access from mobile or laptop with zero waiting time.
Staff repeatedly answers the exact same 50 questions daily, wasting hours of manual productivity.	Automated NLP Knowledge Base answers admissions, syllabus, timings, and campus queries in milliseconds.
Students have no quick private way to check their personal attendance percentage or fee balances.	Secure student login with Registration No. (e.g., ISTU...) delivers personalized real-time stats.
Static noticeboards and clunky PDF circulars are easily missed by students on the go.	Interactive search with quick suggestion pills and spoken audio responses (voice read-aloud).

2. THE THREE CORE PILLARS OF THE SYSTEM



1. Public AI Assistant

Open to any visitor, parent, or applicant. Answers admissions, eligibility, fees, placement stats, and campus facilities with smart typo tolerance.



2. Student Portal

Authenticated zone. When a student enters their Roll No, the bot greets them personally and answers "What is my attendance?", "Do I have fee dues?", etc.



3. Admin Control Panel

Protected back-office. College administrators can add/edit FAQs, monitor live chat logs, track feedback, and export logs to CSV with one click.

HOW IT WORKS UNDER THE HOOD

Step-by-Step Lifecycle of a User Message (From Voice/Typing to Response)

THE LIFECYCLE OF A QUESTION: 7 SIMPLE STEPS

- STEP 1 Student Inputs Query (Typing or Speaking):** The user types in the input box or taps the microphone icon. The browser's native Web Speech API converts speech into text instantly.
- STEP 2 Async HTTP Request (AJAX / Fetch):** JavaScript captures the text, triggers a smooth local audio cue, shows a bouncing "Bot is typing..." indicator, and dispatches a JSON POST request to /chat.
- STEP 3 Session & Student Context Check:** Flask inspects the active browser session. If a student is logged in (e.g. ISTU25010120001), the server first checks if the query asks for personalized data (attendance, marks, fees).
- STEP 4 Intelligent Fuzzy NLP Engine:** If it's a general question, the bot runs the query through our custom NLP matcher in app.py. It cleans tokens and uses Python's difflib.get_close_matches to forgive typos (e.g., "admisn", "timtable").
- STEP 5 Database Lookup & Scoring:** The engine scores matching knowledge-base rows from college_info. The answer with the highest multi-word keyword confidence is selected. If confidence is too low, it triggers a helpful fallback with quick buttons.
- STEP 6 Chat History Logging:** The bot inserts the query, response, student ID, and timestamp into the chat_history table in SQLite and returns the response JSON along with a unique history_id.
- STEP 7 Rendering & Voice Synthesis:** JavaScript receives the response, renders bold text, links, and markdown lists, plays a chime, speaks the response aloud using SpeechSynthesis, and displays Thumbs Up/Down rating buttons!



WHY IS THIS BETTER THAN A GENERIC LLM (LIKE RAW CHATGPT)?

1. **Zero Hallucinations:** The chatbot only gives verified, accurate college data stored in the database.
2. **Zero API Cost & Infinite Speed:** It runs completely on local Python/SQLite without needing paid OpenAI tokens.
3. **Data Privacy:** Student records and internal grades never leave the college server.

TECHNOLOGY STACK & FEATURE HIGHLIGHTS

TECH STACK & SPECIAL FEATURES

Tools Used and the 7 Standout Features that Impress Examiners

1. OUR COMPLETE TECHNOLOGY STACK

Component	Technology	Why We Chose It
Frontend UI	HTML5, CSS3, Modern JavaScript (ES6+)	Blazing fast loading, 100% responsive on mobiles & desktops, zero heavy npm bloat.
Voice I/O	Native Web Speech API (STT & TTS)	Speech recognition and natural voice playback directly inside Chrome/Edge without extra libraries.
Sound Synthesis	Web Audio API (OscillatorNode)	Generates pleasant chime frequencies directly via code without loading heavy MP3 audio files.
Backend Engine	Python 3.11 + Flask Microframework	Lightweight, robust routing, simple session management, and built-in SQLite integration.
Database	SQLite 3 (Embedded Relational DB)	Zero setup, ACID compliant, serverless single-file storage (college_chatbot.db).
Cloud Deployment	Render PaaS (Cloud Linux Container)	Automatic HTTPS/TLS 1.3 encryption, CI/CD GitHub integration, 24/7 global availability.

2. THE 7 SPECIAL FEATURES EXPLAINED SIMPLY

-  **Two-Way Voice Interaction:** Tap the mic to talk to the bot; click the speaker icon on any message to hear it read aloud with smooth speech synthesis.
-  **Personalized Student Intelligence:** When logged in, asking *"what is my attendance?"* returns your actual percentage (e.g. 88.5%) and warns you if below 75%!
-  **Typo-Tolerant NLP Engine:** Students make mistakes when typing quickly. Typing *"admisn eligibility"* or *"feees structre"* still matches the right answer perfectly!
-  **Quality Feedback Loop:** Every answer has Like/Dislike buttons. Clicking a button immediately updates the database row, allowing admins to track bot satisfaction.
-  **Progressive Web App (PWA):** Students can click "Add to Home Screen" on Android/iOS/Windows to install the bot like an official native app with an app icon.
-  **Admin Dashboard & CSV Export:** Administrators have a password-protected portal to add new FAQs on the fly and download the entire chat history as an Excel/CSV file.
-  **Bulletproof Security:** Passwords are never stored in plain text. We use **PBKDF2 with SHA-256** hashing (600,000 rounds) and parameterized SQL queries to prevent SQL Injection.

SYSTEM MODELING & UML DEMYSTIFIED

ALL 8 DIAGRAMS EXPLAINED SIMPLY

How to Explain Each Diagram to Your Teacher in 2 Plain Sentences

- 1 Use Case Diagram (Who uses the system and what can they do?):**
Features the system boundary with two primary actors (**Student** and **Administrator**) on the left, and a dedicated secondary system actor (**Database System - SQLite**) on the right connected to all data operations.
- 2 Class Diagram (What objects exist in the code and how do they connect?):**
Models our 4 core classes (Student, Admin, CollegeInfo, ChatHistory) connected with **100% solid association arrows**, multiplicities (e.g. 1 to 0..*), and explicit role verbs without any dashed lines.
- 3 Object Diagram (A real snapshot of the system running at 10:15 AM):**
Contains **exactly 3 runtime instance boxes** (stu_kanishk : Student, msg_42 : ChatHistory, and faq_101 : CollegeInfo) connected by **solid directional arrows** representing an active query transaction.
- 4 Sequence Diagram (Step-by-step chronological timeline of an interaction):**
Tracks messages passing between Student, Browser, Flask, and SQLite. Most importantly, it includes the **End Chat flow** and formal UML lifeline termination (X) so the examiner sees it has a defined end.
- 5 Activity Diagram (The flowchart of decisions and loops):**
Shows the student workflow: landing on the site, deciding whether to log in, entering a question, looping until satisfied, and cleanly exiting through a double-ring terminal bullseye (O).
- 6 DFD — Data Flow Diagram (Where does data travel in the system?):**
Uses standard Yourdon & DeMarco **circle process symbols** and **solid data flow arrows**. Level 0 features circle 0.0; Level 1 decomposes into 5 circle processes (1.0 to 5.0) and 4 data stores (D1 to D4).
- 7 ERD — Entity-Relationship Diagram (How does the database look?):**
Structured with **pristine left-to-right & top-to-bottom relational flow**: STUDENT (1) → Submits → (N) CHAT_HISTORY (N) ← Answers ← (1) COLLEGE_INFO, audited and managed by ADMIN (top), with zero crossing lines.
- 8 Deployment Diagram (Where does the code physically run in real life?):**
A classic **3-Tier UML 2.0 deployment architecture** with shaded 3D cube nodes: Client Device (Browser), Application Server (Python 3.11 / Flask), and Database Server (SQLite 3), connected by formal communication pipes.

DATABASE ARCHITECTURE & FILE STRUCTURE

DATABASE TABLES & DIRECTORY STRUCTURE

Detailed Schema Breakdown of college_chatbot.db and Code Layout

1. OUR 4 DATABASE TABLES (SQLITE 3)

TABLE 1: students

Stores registered student academic profiles and credentials for personalized queries.

Column	Type	Key	Description
roll_no	TEXT	PK	Unique Registration No (e.g. ISTU25010120001)
name	TEXT	-	Full student name (e.g. Kanishk Singh)
attendance	REAL	-	Current lecture attendance percentage (e.g. 88.5)
fees_due	REAL	-	Pending balance in INR (0.00 if all clear)
cgpa	REAL	-	Cumulative Grade Point Average (out of 10.0)
course / semester	TEXT	-	BBA-CA, Semester III
mentor / exam_date	TEXT	-	Assigned faculty mentor & semester exam date

TABLE 2: college_info

The knowledge-base brain of the chatbot storing categories, keywords, and answers.

Column	Type	Key	Description
id	INTEGER	PK	Auto-incrementing unique identifier
category	TEXT	-	Category name (Admission, Fees, Exams, Hostel)
keywords	TEXT	-	Comma-separated keywords for NLP matching
answer	TEXT	-	Detailed, officially verified institutional response

TABLE 3: chat_history

Auditable transaction log of all conversations and student feedback ratings.

Column	Type	Key	Description
id	INTEGER	PK	Auto-incrementing chat message ID
roll_no	TEXT	FK	References students.roll_no (or NULL for guest)
user_query	TEXT	-	Exact prompt typed or spoken by the student
bot_response	TEXT	-	Answer delivered by the chatbot engine
timestamp	TEXT	-	Timestamp generated by CURRENT_TIMESTAMP
feedback	TEXT	-	Quality rating: 'like' or 'dislike'

TABLE 4: admin

Authentication store for college staff and project administrators.

Column	Type	Key	Description
username	TEXT	PK	Admin username (e.g. 'admin')
password_hash	TEXT	-	PBKDF2 salted hash (never stored in plain text!)

TOP VIVA QUESTIONS & BULLETPROOF ANSWERS

Exactly What to Say When Your Teacher or External Examiner Asks You

Q1: "Is this chatbot using ChatGPT or an external API?"

Answer: "No, sir/ma'am. We intentionally engineered an in-house rule-based Natural Language Processing engine using Python's difflib and keyword scoring. This guarantees 100% data privacy for student records, zero latency, zero cloud API costs, and prevents AI hallucinations."

Q2: "What happens if a student types a spelling mistake like 'admisn'?"

Answer: "Our NLP engine has fuzzy matching enabled with an 80% similarity threshold. When a word like 'admisn' is typed, difflib.get_close_matches automatically detects that it is 85% similar to 'admission', awards a match score, and returns the correct admission guidelines."

Q3: "How does the bot know a student's personal attendance?"

Answer: "When a student logs in using their Registration Number, Flask saves their roll number in a secure server session. When the bot detects personal intent words like 'my attendance' or 'my fees', it intercepts the message and performs an SQL query on the students table for that specific roll number."

Q4: "How is the system secured against hackers or data leaks?"

Answer: "We implemented 3 major security defenses: First, admin passwords are protected using industry-standard PBKDF2 SHA-256 password hashing. Second, all SQL queries use parameterized placeholders (?) to prevent SQL Injection attacks. Third, the live application is served over HTTPS with TLS 1.3 encryption on Render Cloud."

Q5: "How does the voice feature work?"

Answer: "We use the browser's native Web Speech API. For Speech-to-Text, webkitSpeechRecognition captures the student's voice and converts speech to text. For Text-to-Speech, window.speechSynthesis converts the bot's text answer into clear audio output."

Q6: "What is your sequence diagram termination condition?"

Answer: "Our sequence diagram doesn't just hang open; it features an explicit End Chat flow. When the user completes their session or sends a termination command ('bye'), the backend returns a session summary and both the browser and server lifelines are formally destroyed with the UML X destruction mark."

**CLOSING STATEMENT FOR YOUR DEFENSE**

"This project bridges the communication gap between students and the college administration by providing an instant, 24/7, accessible voice and text knowledge assistant that saves faculty time and empowers students with real-time academic records."